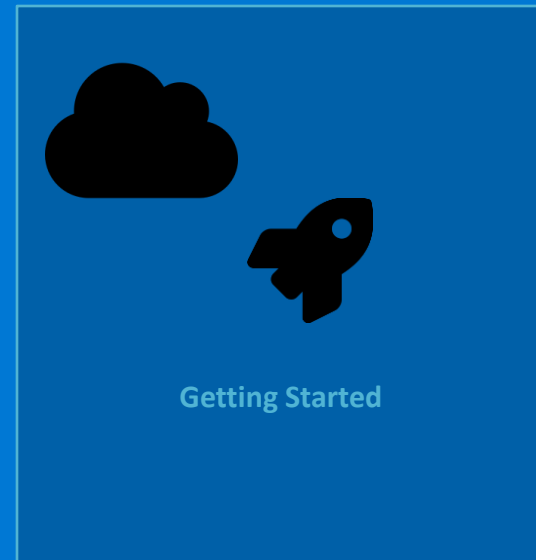


Commencer avec AKS

De zéro à une application déployée sur Azure Kubernetes Service



- 1 Concepts Azure essentiels pour AKS
- 2 AKS vs Kubernetes classique
- 3 Azure Container Registry (ACR)
- 4 Créer un cluster AKS
- 5 Déploiement end-to-end d'une application

Concepts Azure essentiels pour AKS

Les ressources Azure que vous manipulerez autour de votre cluster



Subscription

Conteneur de facturation et d'autorisation de niveau supérieur.
Tous vos services Azure vivent dans une subscription.



Resource Group

Groupe logique de ressources Azure liées.
Cycle de vie partagé — supprimer le RG supprime tout ce qu'il contient.



Virtual Network (VNet)

Réseau privé Azure dans lequel vos ressources communiquent.
AKS déploie ses nœuds dans un subnet du VNet.



Région Azure

Datacenter géographique (France Central, North Europe...).

Choisir la région la plus proche de vos utilisateurs.

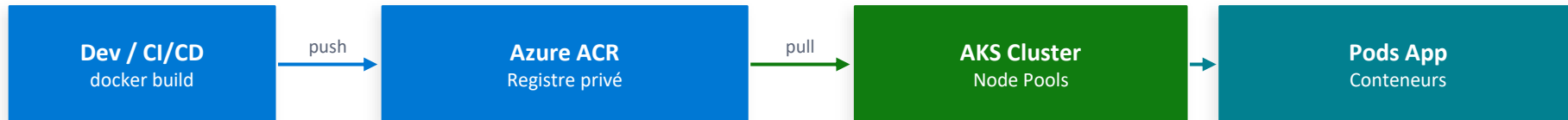
AKS vs Kubernetes classique (self-managed)

Ce qu'Azure prend en charge vs ce qui reste votre responsabilité

Responsabilité	K8s self-managed	AKS (Azure géré)
Plan de contrôle	Vous installez et maintenez	✓ Géré par Azure (gratuit)
Mise à jour K8s	Manuelle — risque élevé	✓ az aks upgrade
Haute disponibilité	Configurer soi-même	✓ Availability Zones
Scalabilité nœuds	Scripts custom	✓ Cluster Autoscaler
Sécurité patches	Votre responsabilité	✓ Auto patch Azure
Intégration Azure	✗ Aucune nativement	✓ AAD, ACR, Monitor, VNet
Coût plan contrôle	VMs à payer	✓ Gratuit
Node pools	Votre gestion	✓ Géré (OS upgrade auto)

Azure Container Registry (ACR)

Stocker, gérer et sécuriser vos images Docker dans Azure



SKUs disponibles

Basic	Stockage : 10 GB Dev/test — pas pour prod
Standard	Stockage : 100 GB Production standard
Premium	Stockage : 500 GB Géo-réplication, Private Link, contenu signé

Commandes essentielles

```
# Créer un registre ACR
az acr create -g myRG -n myACR --sku Standard

# Connecter AKS à ACR
az aks update -g myRG -n myAKS --attach-acr myACR

# Tagger et pousser une image
docker tag myapp:latest myACR.azurecr.io/myapp:v1
az acr login --name myACR
docker push myACR.azurecr.io/myapp:v1

# Lister les images
az acr repository list --name myACR -o table
```

Créer un cluster AKS

Deux méthodes : Azure Portal (GUI) ou Azure CLI (reproductible)

Azure CLI — création complète

1. Créer le Resource Group

```
az group create -n myRG -l francecentral
```

2. Créer le cluster AKS

```
az aks create \
  -g myRG -n myAKS \
  --node-count 3 \
  --node-vm-size Standard_D4s_v3 \
  --zones 1 2 3 \
  --enable-cluster-autoscaler \
  --min-count 3 --max-count 9 \
  --network-plugin azure \
  --attach-acr myACR \
  --enable-addons monitoring
```

3. Récupérer les credentials kubectl

```
az aks get-credentials -g myRG -n myAKS
kubectl get nodes
```

Paramètres clés à comprendre

--node-count

Nombre initial de nœuds (min. 3 pour prod)

--node-vm-size

Taille des VMs (D4s_v3 = 4 vCPU, 16 GB RAM)

--zones 1 2 3

Répartit les nœuds sur 3 datacenters Azure

--network-plugin azure

CNI Azure — IPs VNet directes pour les pods

--attach-acr

Autorise AKS à puller les images depuis ACR

--enable-addons monitoring

Active Azure Monitor Container Insights

Déploiement end-to-end d'une application sur AKS

Du code source à l'URL publique accessible — en 5 étapes

1

Build & Push image Docker vers ACR

```
docker build -t myACR.azurecr.io/myapp:v1 .  
docker push myACR.azurecr.io/myapp:v1
```

✓ Résultat :

Image disponible dans ACR

2

Créer le Deployment K8s

```
kubectl create deployment myapp \  
  --image=myACR.azurecr.io/myapp:v1 \  
  --replicas=3
```

✓ Résultat :

3 pods Running sur les nœuds

3

Exposer via un Service (ClusterIP)

```
kubectl expose deployment myapp \  
  --port=80 --target-port=8080
```

✓ Résultat :

Service routable en interne

4

Créer l'Ingress HTTPS

```
kubectl apply -f ingress.yaml  
# host: myapp.example.com, TLS cert-manager
```

✓ Résultat :

HTTPS actif + cert Let's Encrypt

5

Vérifier et tester

```
kubectl get pods,svc,ingress  
curl -I https://myapp.example.com
```

✓ Résultat :

App accessible sur Internet ✓

TP — Déployer votre première application sur AKS

Durée : 45 min | Prérequis : cluster AKS créé, az CLI, kubectl, ACR attaché

1

Créer et pousser une image vers ACR

```
docker build -t myACR.azurecr.io/hello-aks:v1 .  
az acr login --name myACR  
docker push myACR.azurecr.io/hello-aks:v1
```

✓ Validation :

Image visible : `az acr repository list --name myACR`

2

Déployer l'application (3 réplicas)

```
kubectl create deployment hello-aks --image=myACR.azurecr.io/hello-aks:v1 --  
replicas=3  
kubectl get pods -w # Attendre Running
```

✓ Validation :

3 pods en status Running sur des nœuds différents

3

Exposer via LoadBalancer et tester

```
kubectl expose deployment hello-aks --type=LoadBalancer --port=80 --target-port=8080  
kubectl get svc -w # Attendre IP publique  
curl http://<EXTERNAL-IP>
```

✓ Validation :

Réponse HTTP 200 depuis l'IP publique Azure

4

Scaler et observer

```
kubectl scale deployment hello-aks --replicas=6  
kubectl get pods -o wide # Pods sur plusieurs nœuds  
kubectl rollout status deployment/hello-aks
```

✓ Validation :

6 pods répartis sur les nœuds du cluster

⚠ Penser à supprimer le Service LoadBalancer après le TP — `az network public-ip list -g MC_myRG_myAKS` — pour éviter toute facturation.